

Course Outline (Syllabus of the Course)

Course Code: CS3704 Name of the Programme: BTech (H) Name of the Course: Compiler Design Semester: VI		
Course credit	No. of hours per week (L : T : P)	Total no. of Teaching hours
03	3:0:0	45
Course Objectives:		
a. Introduce the fundamental concepts and phases of compiler design, including lexical, syntax, and semantic analysis.		
b. Explain parsing techniques, such as top-down and bottom-up approaches, and their role in syntax analysis.		
c. Teach syntax-directed translation, intermediate code generation, and explore runtime structures like symbol tables and memory management.		
d. Provide an overview of web and IP security principles to support the development of secure systems.		

Module – 1	No. of hours
Introduction to Compiler	9
Definition of compiler, interpreter and its differences, the phases of a compiler, role of lexical analyzer, regular expressions, finite automata, finite automata to regular expressions, bootstrapping, Lexical Analyzer Generator.	

Module – 2	No. of hours
Parsing Techniques	9
Context free grammar, derivations, parse trees, ambiguity, eliminating ambiguity, elimination of left recursion, left factoring, top down parsing: backtracking, recursive descent parsing, predictive parsers, LL(1) grammars, bottom up parsing: LR parsers, Simple LR, Canonical LR(CLR) and Look Ahead LR (LALR) parsers, Operator precedence parsing.	

Module – 3	No. of hours
Syntax-Directed Translation	9
Syntax directed definition, construction of syntax trees, S-attributed and L-attributed definitions, Syntax-directed Translation schemes and applications, Intermediate code, postfix notation, Parse trees & syntax trees, three address code, quadruple & triples, Translation of simple statements and control flow statements.	

Module – 4	No. of hours
Symbol Tables & Run-Time Environments	9
Symbol Tables: Data structure for symbols tables, representing scope information. Run-Time Environments: Implementation of simple stack allocation scheme. Error Detection & Recovery: Lexical Phase errors, syntactic phase errors and semantic errors.	

Module – 5	No. of hours
Code Optimization & Generation	9
Design Issues, the Target Language. Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, Code Generator. Code optimization: Machine-Independent Optimizations, Loop optimization, DAG representation of basic blocks, value numbers and algebraic laws, Data-Flow analysis: constant propagation, liveness analysis. Note: Developing a simple parser using Lex/Yacc will be given as a part of assignment.	

Course Outcomes: After completing the course, the students will be able to: -	
CO 1	Understand and explore the fundamental phases of compiler design, including lexical analysis, syntax analysis, and semantic analysis.
CO 2	Apply top-down and bottom-up parsing techniques, such as recursive descent, LL(1), and LR parsers, for syntactic analysis.
CO 3	Develop syntax-directed translation schemes and generate intermediate code, while understanding runtime structures like symbol tables and their implementation.
CO 4	Apply IP and Web security principles to design and evaluate secure systems in real-world networking scenarios.

Text Books	
1.	A. V. Aho, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, Pearson Education. ISBN: 978-9332518667.
2.	V. Raghavan, Principles of Compiler Design, New Delhi, India: TMH (Tata McGraw-Hill), ISBN: 978-0070144712.

Reference Books	
1.	K. Loudon, Compiler Construction: Principles and Practice, Boston, MA, USA: Course Technology Inc., ISBN: 978-0534939724.
2.	C. Fischer and R. LeBlanc, Crafting a Compiler with C, Pearson Education, ISBN: 978-8131708132.